Bresenham 画直线与画圆算法笔记

一、核心思想

理想的直线和圆是连续曲线，而屏幕只能绘制整数坐标上的像素。

Bresenham 算法的思路是：

1. 沿曲线的主要方向，每次推进一个像素。
2. 在少数几个候选像素中，选择更贴近理想曲线的那个。
3. 用一个**判断量**记录偏差，通过整数加减更新，避免每一步重新计算乘法、除法或平方。

判断量通常叫 `err`、`p` 或 `d`。**它们并不直接等于像素到曲线的几何距离，其含义必须结合定义理解**。

二、Bresenham 画直线

1. 将任意方向归一化

给定起点 (x_0, y_0) 和终点 (x_1, y_1)，定义：

$$\mathrm{d}x = |x_1 - x_0|, \qquad \mathrm{d}y = |y_1 - y_0|$$

实际坐标的推进方向由下式控制：

```
int sx = x0 < x1 ? 1 : -1;
int sy = y0 < y1 ? 1 : -1;
```

为了推导方便，将起点平移到原点，并将坐标轴朝向终点。这样只需研究：

$$(0,0) \longrightarrow (\mathrm{d}x, \mathrm{d}y)$$

下文推导中的 (x, y) 指这个归一化坐标系中的位置。

2. 定义偏差

理想直线满足：

$$\mathrm{d}x \cdot y - \mathrm{d}y \cdot x = 0$$

定义：

$$\boxed{F = \mathrm{d}x \cdot y - \mathrm{d}y \cdot x}$$

当 $\mathrm{d}x > 0$ 时：

$$F = \mathrm{d}x \left(y - \frac{\mathrm{d}y}{\mathrm{d}x} x \right)$$

因此：

- $F = 0$：像素在直线上。
- $F > 0$：像素在直线上方。

- $F<0$：像素在直线下方。

这个偏差有非常简单的更新规则：

$$\boxed{x\text{增加}1 \quad \Rightarrow \quad F \leftarrow F - \mathrm{d}y}$$

$$\boxed{y\text{增加}1 \quad \Rightarrow \quad F \leftarrow F + \mathrm{d}x}$$

3. 平缓直线：每列选择一个像素

当 dx≥dy 且 dx>0 时，每次 x 增加 1，候选像素为：

$$A = (x+1, y), \qquad B = (x+1, y+1)$$

两个候选的中点高度为 $y + \frac{1}{2}$。

下一列中，理想直线的高度为：

$$y_{\text{理想}} = \frac{\mathrm{d}y}{\mathrm{d}x}(x+1)$$

若理想高度达到或超过中点，选择 B，让 y 也增加 1：

$$\frac{\mathrm{d}y}{\mathrm{d}x}(x+1) \geq y + \frac{1}{2}$$

整理得到：

$$\boxed{F \leq \mathrm{d}y - \frac{\mathrm{d}x}{2}}$$

否则选择 A，保持 y 不变。

等号表示两个像素同样接近理想直线；这里约定选择 B。

4. 陡峭直线：每行选择一个像素

当 dy≥dx 且 dy>0 时，每次 y 增加 1，候选像素为：

$$A = (x, y+1), \qquad B = (x+1, y+1)$$

如果下一行中理想直线的横坐标达到或超过 $x + \frac{1}{2}$，就让 x 也增加 1：

$$\frac{\mathrm{d}x}{\mathrm{d}y}(y+1) \geq x + \frac{1}{2}$$

整理得到：

$$\boxed{F \geq \frac{\mathrm{d}y}{2} - \mathrm{d}x}$$

否则 x 不变。

5. 引入 `err`，统一两种情况

定义：

$$\boxed{\mathrm{err} = F + \mathrm{d}x - \mathrm{d}y}$$

于是两个判断条件分别变为：

$$\boxed{\text{推进}x\text{：}\quad 2\text{err}\geq -\text{dy}}$$

$$\boxed{\text{推进}y\text{：}\quad 2\text{err}\leq \text{dx}}$$

因为起点处 $F=0$，所以：

$$\boxed{\text{err}_0=\text{d}x-\text{dy}}$$

`err` 与 F 只相差常数，更新方式也相同：

```
推进 x: err -= dy;
推进 y: err += dx;
```

6. 为什么不必先判断平缓还是陡峭?

关键是：**主要方向的推进条件会自动成立**。

平缓时，推进 x 的条件必定成立

当前像素是本列最近的像素，所以纵向偏差不超过半个像素：

$$-\frac{\text{d}x}{2}\leq F\leq\frac{\text{d}x}{2}$$

因此：

$$2\text{err}=2F+2\text{dx}-2\text{dy}\geq\text{dx}-2\text{dy}\geq-\text{dy}$$

最后一步使用了 $\text{dx}\geq\text{dy}$。

所以 x 每轮必定推进，y 则由中点判断决定。

陡峭时，推进 y 的条件必定成立

当前像素是本行最近的像素，所以横向偏差不超过半个像素，从而：

$$-\frac{\text{dy}}{2}\leq F\leq\frac{\text{dy}}{2}$$

因此：

$$2\text{err}=2F+2\text{dx}-2\text{dy}\leq2\text{dx}-\text{dy}\leq\text{dx}$$

最后一步使用了 $\text{dy}\geq\text{dx}$。

所以 y 每轮必定推进，x 则由中点判断决定。

这些偏差范围可以归纳证明：起点偏差为 0；每轮的中点选择又保证下一点是下一列或下一行中最近的像素。

7. 完整代码

```
#include <cstdlib>

void plotLine(int x0, int y0, int x1, int y1)
{
```

```
        int dx = std::abs(x1 - x0);
        int dy = std::abs(y1 - y0);

        int sx = x0 < x1 ? 1 : -1;
        int sy = y0 < y1 ? 1 : -1;

        int err = dx - dy;

        for (;;) {
            setPixel(x0, y0);

            if (x0 == x1 && y0 == y1)
                break;

            int e2 = 2 * err;

            if (e2 >= -dy) {
                err -= dy;
                x0 += sx;
            }

            if (e2 <= dx) {
                err += dx;
                y0 += sy;
            }
        }
    }
```

该写法支持水平线、竖直线、任意象限，以及起终点重合的情况。

为什么必须保存 `e2`?

两个判断都针对**本轮开始时的同一个像素**推导，必须使用同一份原始误差。

第一个分支可能修改 `err`，第二个判断仍应使用保存的 `e2`。

为什么不能使用 `else if`?

因为某一轮可能同时推进 x 和 y，即沿对角方向移动一个像素。

8. 示例：从 (0,0) 到 (5,1)

初始化：

$$\mathrm{d}x = 5, \qquad \mathrm{dy} = 1, \qquad \mathrm{err} = 4$$

当前像素	err	e2	本轮移动
(0,0)	4	8	只推进 x
(1,0)	3	6	只推进 x
(2,0)	2	4	同时推进 x、y
(3,1)	6	12	只推进 x
(4,1)	5	10	只推进 x
(5,1)	4	—	到达终点

最终像素：

(0,0) (1,0) (2,0) (3,1) (4,1) (5,1)

9. 为什么不能把两个阈值都换成 0?

下面的写法与标准算法不等价:

```
if (err >= 0) {
    err -= dy;
    x += sx;
}
if (err <= 0) {
    err += dx;
    y += sy;
}
```

它同时改变了:

1. 判断阈值。
2. 第二个判断所使用的误差时刻。

对于 `(0,0)` → `(5,1)`，它会选中 `(3,0)`，而不是 `(3,1)`。

理想直线在 `x=3` 时的高度是 `0.6`，显然 `(3,1)` 更近。因此这不是平局处理方式不同，而是选点有误。

三、Bresenham 画圆

1. 利用八重对称性

设圆心在原点，半径为非负整数 r:

$$x^2 + y^2 = r^2$$

只需计算从 $(0, r)$ 到 $x = y$ 附近的八分之一圆弧。

这一段圆弧比较平缓。每次 x 增加 1，y 保持不变或减 1，因此候选像素为:

$$E = (x+1, y), \qquad \mathrm{SE} = (x+1, y-1)$$

每算出一个 (x, y)，即可绘制八个对称点:

$$(\pm x, \pm y), \qquad (\pm y, \pm x)$$

圆心为 (c_x, c_y) 时，将这些坐标加上圆心即可。

2. 定义圆函数

令:

$$\boxed{F(u, v) = u^2 + v^2 - r^2}$$

则:

- $F < 0$: 点在圆内。
- $F = 0$: 点在圆上。
- $F > 0$: 点在圆外。

注意，F 是“到圆心距离的平方减去半径平方”，不是到圆周的实际距离。

3. 用中点选择像素

两个候选的中点是:

$$M=\left(x+1,y-\frac{1}{2}\right)$$

定义:

$$\boxed{q=F(M)=(x+1)^2+\left(y-\frac{1}{2}\right)^2-r^2}$$

判断规则为:

条件	几何含义	选择
$q<0$	中点在圆内，圆弧位于中点上方	E，y 不变
$q>0$	中点在圆外，圆弧位于中点下方	SE，y 减 1

这里比较的是同一列中两个候选像素的纵向偏差。

4. 中点圆算法的整数形式：`p = 1-r`

起点为 $(0,r)$，所以:

$$q_0=1+\left(r-\frac{1}{2}\right)^2-r^2=\frac{5}{4}-r$$

定义整数判断量:

$$\boxed{p=q-\frac{1}{4}}$$

于是:

$$\boxed{p_0=1-r}$$

由于整数坐标下 q 总是"整数加 $1/4$":

$$q<0\Longleftrightarrow p<0,\qquad q>0\Longleftrightarrow p\geq 0$$

因此减去 $1/4$ 不会改变像素选择。特别地，**p 等于 0 时，q 等于 1/4，应选择 SE**。

5. 推导增量更新

以下公式中的 x、y 都是**移动前的坐标**。

选择 E：新像素为（`x+1,y`）

下一轮的中点变为:

$$M'=\left(x+2,y-\frac{1}{2}\right)$$

因此:

$$\Delta p=(x+2)^2-(x+1)^2=2x+3$$

即:

$$\boxed{p\leftarrow p+2x+3}$$

选择 SE：新像素为（x+1,y-1）

下一轮的中点变为：

$$M' = \left(x+2, y-\frac{3}{2}\right)$$

因此：

$$\begin{aligned}\Delta p &= (x+2)^2 - (x+1)^2 \\ &\quad + \left(y-\frac{3}{2}\right)^2 - \left(y-\frac{1}{2}\right)^2 \\ &=\end{aligned}$$

即：

$$\boxed{p \leftarrow p + 2(x-y) + 5}$$

核心循环为：

```
int x = 0;
int y = r;
int p = 1 - r;

while (x <= y) {
    plot8(cx, cy, x, y);

    if (p < 0) {
        p += 2 * x + 3;
    } else {
        p += 2 * (x - y) + 5;
        --y;
    }
    ++x;
}
```

6. Bresenham 形式：为什么是 d = 3-2r?

常见的 Bresenham 圆算法定义另一判断量：

$$\boxed{d = F(E) + F(\mathrm{SE})}$$

在起点 $(0, r)$：

$$F(E) = F(1, r) = 1$$

$$F(\mathrm{SE}) = F(1, r-1) = 2 - 2r$$

所以：

$$\boxed{d_0 = 3 - 2r}$$

展开 d 可得：

$$d = 2(x+1)^2 + 2y^2 - 2y + 1 - 2r^2$$

而：

$$p = (x+1)^2 + y^2 - y - r^2$$

因此：

$$\boxed{d = 2p + 1}$$

由于 p 是整数：

$$p < 0 \Longleftrightarrow d < 0, \qquad p \geq 0 \Longleftrightarrow d > 0$$

所以两种形式选择完全相同的像素。d 始终是奇数，不会等于 0。

同时，d 的变化量是 p 的两倍：

项目	中点形式 p	Bresenham 形式 d
初始化	`1 - r`	`3 - 2*r`
选择 E	`p < 0`	`d < 0`
选择 E 后增加	`2*x + 3`	`4*x + 6`
选择 SE 后增加	`2*(x-y) + 5`	`4*(x-y) + 10`

初始化、判断规则和增量公式必须配套使用，不能只替换初始值。

7. 完整代码

```
void plot8(int cx, int cy, int x, int y)
{
    setPixel(cx + x, cy + y);
    setPixel(cx - x, cy + y);
    setPixel(cx + x, cy - y);
    setPixel(cx - x, cy - y);

    setPixel(cx + y, cy + x);
    setPixel(cx - y, cy + x);
    setPixel(cx + y, cy - x);
    setPixel(cx - y, cy - x);
}

void plotCircle(int cx, int cy, int r)
{
    if (r < 0)
        return;

    int x = 0;
    int y = r;
    int d = 3 - 2 * r;

    while (x <= y) {
        plot8(cx, cy, x, y);

        if (d < 0) {
            d += 4 * x + 6;
        } else {
            d += 4 * (x - y) + 10;
            --y;
        }

        ++x;
    }
```

```
}
```

使用时注意：

- `x <= y` 保证包含对角线上的点；越过对角线后，剩余圆弧由对称性覆盖。
- 增量使用旧的 x、y，所以应先更新 d，再修改坐标。
- 对称轴上的点会重复绘制；普通像素赋值不受影响，但透明混合或计数时需要去重。
- r 为 0 时，只会绘制圆心，虽然上述实现会重复赋值。

8. 示例：半径 r=5

初始化：

$$x = 0, \qquad y = 5, \qquad d = 3 - 10 = -7$$

当前像素	d	选择	更新后的 d	下一像素
(0,5)	-7	E	-1	(1,5)
(1,5)	-1	E	9	(2,5)
(2,5)	9	SE	7	(3,4)
(3,4)	7	SE	13	(4,3)

到达 (4,3) 时已有 `x > y`，不再绘制这一八分之一圆弧。它对应的对称点已由 (3,4) 覆盖。

实际计算的基本点为：

(0,5) (1,5) (2,5) (3,4)

再通过八重对称得到整个圆。

四、两种算法的联系与区别

比较项	直线	圆
隐式方程	$\mathrm{d}x\, y - \mathrm{d}y\, x = 0$	$x^2 + y^2 - r^2 = 0$
选择依据	理想直线在候选中点哪一侧	候选中点在圆内还是圆外
主要方向	根据斜率自动表现为 x 或 y	基本圆弧中始终推进 x
另一方向	不变或推进一步	y 不变或减少一步
判断量更新	加减固定的 dx、dy	加上关于当前 x、y 的一次式
对称性	用 sx、sy 处理方向	计算八分之一，再生成对称点
时间复杂度	$O(\max(\mathrm{d}x, \mathrm{d}y) + 1)$	$O(r + 1)$
额外空间	$O(1)$	$O(1)$

直线方程是一次式， 移动一步后的差分是常数； 圆方程是二次式， 移动一步后的差分是一次式。因此它们都能避免逐点重新计算原方程。

以上代码假设坐标及中间运算不会超出 `int` 范围；较大坐标应使用足够宽的整数类型，并在做差之前完成类型转换。